

Database & Architecture Migration — Technical Design

Current State (Problems to Solve)

Problem	Impact
JSON files on disk as "database"	No querying, filtering, indexing, or joins
Two backends reading the same folder	Race conditions when scraper writes while .NET reads
No persistence for Leads	Lost on restart
Full directory scan on every API call	O(n) disk I/O, slow at scale
No caching layer	Every frontend request hits disk
Python Flask + .NET doing redundant work	Two backends serving overlapping data

Recommended Database

Primary: PostgreSQL

Why PostgreSQL over alternatives:

Option	Verdict	Reason
PostgreSQL	Recommended	JSONB columns handle variable RERA fields natively; full SQL for SRO analytics; free, battle-tested, great .NET support via Npgsql
SQLite	Single-writer lock breaks concurrent scraper + API writes	
MongoDB	Good for flexible schema but overkill; adds ops complexity; no JOIN support for SRO analytics	
MySQL	No JSONB; weaker full-text search	
SQL Server	Licensing cost; no benefit over PostgreSQL here	

Key reason: RERA project data has a variable, deeply nested structure. PostgreSQL's `JSONB` column lets us store the raw scraped blob while also indexing specific fields (`pin\_code`, `district`, `project\_status`) for fast queries — best of both worlds.

Secondary: Redis (optional, phase 2)

For caching the `/api/projects` response (43 projects, read frequently, updated infrequently). Not required for phase 1.

High-Level Architecture

```
```mermaid
graph TD
 subgraph Scrapers["Python Scrapers (port 5000)"]
 A[rera_detail_scraper] --> DB
 B[sro_transaction_scraper] --> DB
 C[rr_scraper] --> DB
 end

 subgraph DB["PostgreSQL Database"]
 T1[(projects)]
 T2[(sro_transactions)]
 T3[(unit_rates)]
 T4[(leads)]
 T5[(scrape_runs)]
 end

 subgraph API[".NET API (port 5001)"]
 D[ProjectController]
 E[SroController]
 F[LeadController]
 G[AdminController]
 end

 subgraph Frontend["Angular Frontend (port 4200)"]
 H[Properties List]
 I[Admin Panel]
 J[Market Analytics]
 end
```

end

DB --> API

API --> Frontend

I --> G

G -->|trigger scrape| A

G -->|trigger scrape| B

G -->|trigger scrape| C

...

## Key Architectural Changes

1. **Python Flask** role narrows to **scraper orchestration only** — it writes to PostgreSQL instead of JSON files. It no longer serves any data to the frontend.
2. **.NET API** becomes the **single data API** — all frontend calls go through it.
3. **Flask's `/api/projects` endpoint is removed** — .NET owns that entirely.
4. The shared `scraped\_projects/` folder is eliminated.

---

## Database Schema (Low-Level Design)

Table: `projects`

```sql

CREATE TABLE projects (

id TEXT PRIMARY KEY, -- sanitized project name (folder name)

project_name TEXT NOT NULL,

project_status TEXT,

project_type TEXT,

district TEXT,

mandal TEXT,

locality TEXT,

pin_code TEXT,

village TEXT,

approved_date DATE,

completion_date DATE,

```

revised_completion_date DATE,
total_area_sqmt NUMERIC,
net_area_sqmt NUMERIC,
built_up_area_sqmt NUMERIC,
mortgage_area_sqmt NUMERIC,
promoter_name TEXT,
org_type TEXT,
bank_name TEXT,
branch_name TEXT,
plan_approval_number TEXT,
survey_number TEXT,
is_msb BOOLEAN DEFAULT FALSE,
has_litigation BOOLEAN DEFAULT FALSE,
total_flats INTEGER,
total_booked INTEGER,
saleable_area_sqmt NUMERIC,
raw_data JSONB NOT NULL, -- full scraped blob (view_page_data.json contents)
pricing JSONB, -- admin-entered pricing.json
available_documents TEXT[],
scraped_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes for common query patterns
CREATE INDEX idx_projects_pin_code ON projects(pin_code);
CREATE INDEX idx_projects_district ON projects(district);
CREATE INDEX idx_projects_status ON projects(project_status);
CREATE INDEX idx_projects_locality ON projects(locality);
CREATE INDEX idx_projects_raw_data ON projects USING GIN(raw_data); -- JSONB full
search
```

```

Table: `sro\_transactions`

```

```sql
CREATE TABLE sro_transactions (
id SERIAL PRIMARY KEY,
sro_name TEXT,

```

```
district TEXT,  
village TEXT,  
apartment TEXT,  
flat_no TEXT,  
reg_date DATE,  
quarter TEXT, -- e.g. "2024-Q2"  
mkt_value BIGINT,  
cons_value BIGINT,  
price_per_sqft NUMERIC,  
scraped_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE INDEX idx_sro_village ON sro_transactions(village);  
CREATE INDEX idx_sro_quarter ON sro_transactions(quarter);  
CREATE INDEX idx_sro_apartment ON sro_transactions(apartment);  
...
```

Table: `unit\_rates`

```
```sql  
CREATE TABLE unit_rates (
id SERIAL PRIMARY KEY,
district TEXT,
mandal TEXT,
locality TEXT,
search_type TEXT DEFAULT 'apartment', -- 'apartment' | 'land'
unit_rate_sqft NUMERIC,
scraped_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_unit_rates_mandal ON unit_rates(mandal);
CREATE INDEX idx_unit_rates_district ON unit_rates(district);
...
```

Table: `leads`

```
```sql
```

```

CREATE TABLE leads (
id SERIAL PRIMARY KEY,
name TEXT NOT NULL,
email TEXT NOT NULL,
mobile TEXT NOT NULL,
area_of_interest TEXT,
project_name TEXT,
project_id TEXT REFERENCES projects(id),
device_fingerprint TEXT,
source TEXT DEFAULT 'property_detail_page',
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

Table: `scrape\_runs`

```

```sql
CREATE TABLE scrape_runs (
id SERIAL PRIMARY KEY,
scraper TEXT NOT NULL, -- 'rera', 'sro', 'rr'
status TEXT NOT NULL, -- 'running', 'completed', 'failed'
started_at TIMESTAMPTZ DEFAULT NOW(),
finished_at TIMESTAMPTZ,
total INTEGER DEFAULT 0,
completed INTEGER DEFAULT 0,
errors JSONB DEFAULT '[]'
);
```

```

---

## Data Flow

Scrape → Database Flow

...

## Python Scraper

```
|
|─ rera_detail_scraper.py
| scrapes view_page_data HTML
| → INSERT INTO projects (id, project_name, ..., raw_data)
| ON CONFLICT (id) DO UPDATE SET raw_data=..., updated_at=NOW()
|
|─ sro_transaction_scraper.py
| → INSERT INTO sro_transactions (...)
| ON CONFLICT DO NOTHING (idempotent)
|
|─ rr_scraper.py
→ INSERT INTO unit_rates (...)
(truncate + re-insert on each run)
...

```

## API Read Flow

```
...
Angular Frontend
|
GET /api/projects
|
.NET ProjectController
|
ProjectRepository (Npgsql)
|
SELECT id, project_name, district, locality, pin_code,
project_status, total_flats, booking_percent,
raw_data->'Floor Breakdown' AS floor_breakdown,
pricing
FROM projects
WHERE pin_code = @pinCode ← optional filter
ORDER BY project_name
|
PostgreSQL
...

```

---

## .NET Repository Changes

### Current (file-based)

```
```csharp
// Reads every subdirectory and parses JSON files
var directories = Directory.GetDirectories(_scrapedProjectsPath);
foreach (var subdir in directories) { ... File.ReadAllTextAsync(jsonPath) ... }
```
```

### New (PostgreSQL via Npgsql + Dapper)

```
```csharp
public async Task<IEnumerable<Dictionary<string, object>>> GetAllProjectsAsync()
{
    const string sql = @"
SELECT id, project_name, project_status, project_type,
district, mandal, locality, pin_code,
total_flats, total_booked, saleable_area_sqmt,
raw_data, pricing, available_documents,
scraped_at
FROM projects
ORDER BY project_name";

    await using var conn = new NpgsqlConnection(_connectionString);
    var rows = await conn.QueryAsync<dynamic>(sql);
    return rows.Select(MapToProjectDict);
}

public async Task<Dictionary<string, object>?> GetProjectByIdAsync(string id)
{
    const string sql = "SELECT * FROM projects WHERE id = @id";
    await using var conn = new NpgsqlConnection(_connectionString);
    var row = await conn.QuerySingleOrDefaultAsync<dynamic>(sql, new { id });
    return row == null ? null : MapToProjectDict(row);
}
```



```
}  
'''
```

Python Scraper Write (psycopg2)

```
```python  
import psycopg2, json

def save_project_to_db(conn, project_id: str, data: dict):
 with conn.cursor() as cur:
 cur.execute("""
INSERT INTO projects (
id, project_name, project_status, district, mandal,
locality, pin_code, total_flats, total_booked, raw_data, scraped_at
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, NOW())
ON CONFLICT (id) DO UPDATE SET
raw_data = EXCLUDED.raw_data,
updated_at = NOW()
""", (
project_id,
data.get('Project Name'),
data.get('Project Status'),
data.get('District'),
data.get('Mandal'),
data.get('Locality'),
data.get('Pin Code'),
data.get('totalFlats', 0),
data.get('totalBookedFlats', 0),
json.dumps(data),
))
 conn.commit()
'''
```

---

## Migration Path (Phased)

## Phase 1 — Introduce PostgreSQL alongside JSON files (no breaking changes)

- Install PostgreSQL locally / on server
- Add `psycopg2` to Python requirements
- Modify scrapers to write to DB **in addition** to JSON files
- Add `Npgsql` + `Dapper` to .NET project
- Add new `PostgresProjectRepository` implementing existing `IProjectRepository`
- Toggle between file-based and DB-based via `appsettings.json` flag
- Verify data parity

## Phase 2 — Switch .NET to read from PostgreSQL exclusively

- Flip the DI registration to `PostgresProjectRepository`
- Remove `ProjectRepository` (file-based)
- Remove `ScrapedProjectsPath` from appsettings

## Phase 3 — Remove JSON file writing from scrapers

- Remove `json.dump(...)` calls in scrapers
- Delete `scraped\_projects/` folder from codebase
- Remove Flask's `/api/projects` endpoint (dead code)

## Phase 4 — Add Redis cache (optional)

- Cache `GET /api/projects` response for 5 minutes in Redis
- Invalidate on scrape completion

---

## NuGet / pip Packages Required

**\*\* .NET additions:**

```xml

<PackageReference Include="Npgsql" Version="8.0.5" />

<PackageReference Include="Dapper" Version="2.1.35" />

```

**\*\* Python additions:**

```

```
psycopg2-binary==2.9.10
```

```
```
```

```

```

## Connection String Configuration

```
appsettings.json:
```

```
```json
```

```
{
```

```
"ConnectionStrings": {
```

```
"DefaultConnection": "Host=localhost;Port=5432;Database=hydurban;Username=hydurban_app;Password=CHANGE_IN_PRODUCTION"
```

```
},
```

```
"FeatureFlags": {
```

```
"UsePostgresRepository": false
```

```
}
```

```
}
```

```
```
```

```
Python (.env or scrape_preferences.json extension):
```

```
```json
```

```
{
```

```
"db_connection": "postgresql://hydurban_app:password@localhost:5432/hydurban"
```

```
}
```

```
```
```

```

```

## Correctness Properties (for Property-Based Testing)

1. **\*\*Idempotency\*\*** — Running the scraper twice for the same project must produce identical DB state (no duplicates, no data loss via ``ON CONFLICT DO UPDATE``)
2. **\*\*Field completeness\*\*** — Every project row must have ``id``, ``project_name``, and ``raw_data`` non-null
3. **\*\*Round-trip fidelity\*\*** — Data written by Python scraper, read by .NET API must produce the same field values as the current JSON-file pipeline

4. **Filter correctness** — ``GetProjectsByPinCode("500075")`` must return only rows where ``pin_code` = '500075'`
5. **No data loss on migration** — Row count in PostgreSQL after Phase 1 migration equals directory count in ``scraped_projects/``

---

## Part 2: User Auth & Personalisation Features

### New Features Scope

| Feature            | Description                                           |
|--------------------|-------------------------------------------------------|
| Sign Up            | Email + password registration with email verification |
| Sign In            | JWT-based login, refresh token support                |
| Forgot Password    | Email OTP / reset link flow                           |
| Profile            | View and edit name, mobile, avatar, preferences       |
| Saved Properties   | Bookmark a project; view/remove saved list            |
| Saved Searches     | Save a search filter set; re-run it anytime           |
| Comparison Results | Persist side-by-side project comparisons              |
| Favorites          | Heart/unfavorite a project; dedicated favorites view  |

> Note: **Favorites** and **Saved Properties** are intentionally separate — Favorites is a quick heart-button, Saved Properties is a deliberate "I want to revisit this" bookmark. Both store ``project_id`` references, but rendered differently in the UI.

---

## Architecture Changes for User System

```
```mermaid
graph TD
  subgraph Auth["Auth Layer (.NET)"]
    AU[UserController /api/auth/*]
    AS[UserService]
    AR[UserRepository]
  end
```

```

subgraph UserData["User Features (.NET)"]
  UC[UserController /api/user/*]
  UDS[UserDataService]
  UDR[UserDataRepository]
end

```

```

subgraph DB["PostgreSQL"]
  U[(users)]
  RT[(refresh_tokens)]
  PWT[(password_reset_tokens)]
  SP[(saved_properties)]
  SS[(saved_searches)]
  CR[(comparison_results)]
  FAV[(favorites)]
end

```

```

subgraph Email["Email Service"]
  ES[SmtpEmailService / SendGrid]
end

```

```

AU --> AS --> AR --> U
AU --> ES
UC --> UDS --> UDR
UDR --> SP & SS & CR & FAV
...

```

Database Schema — User Tables

Table: `users`

```

```sql
CREATE TABLE users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 email TEXT NOT NULL UNIQUE,

```

```
password_hash TEXT NOT NULL, -- BCrypt hash
full_name TEXT NOT NULL,
mobile TEXT,
avatar_url TEXT,
is_verified BOOLEAN DEFAULT FALSE,
is_active BOOLEAN DEFAULT TRUE,
role TEXT DEFAULT 'user', -- 'user' | 'admin'
email_verified_at TIMESTAMPTZ,
last_login_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_users_email ON users(email);
```
```

Table: `refresh\_tokens`

```
```sql
CREATE TABLE refresh_tokens (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 token_hash TEXT NOT NULL UNIQUE, -- SHA-256 hash of the token
 expires_at TIMESTAMPTZ NOT NULL,
 revoked_at TIMESTAMPTZ,
 device_info TEXT, -- browser / device hint
 created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_refresh_tokens_user ON refresh_tokens(user_id);
CREATE INDEX idx_refresh_tokens_token ON refresh_tokens(token_hash);
```
```

Table: `password\_reset\_tokens`

```
```sql
CREATE TABLE password_reset_tokens (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
token_hash TEXT NOT NULL UNIQUE,
expires_at TIMESTAMPTZ NOT NULL,
used_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

Table: `email\_verification\_tokens`

```

```sql
CREATE TABLE email_verification_tokens (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
token_hash TEXT NOT NULL UNIQUE,
expires_at TIMESTAMPTZ NOT NULL,
used_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

Table: `saved\_properties`

```

```sql
CREATE TABLE saved_properties (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
notes TEXT, -- optional user note
created_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(user_id, project_id)
);

CREATE INDEX idx_saved_props_user ON saved_properties(user_id);
```

```

Table: `favorites`

```
```sql
CREATE TABLE favorites (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
 created_at TIMESTAMPTZ DEFAULT NOW(),
 UNIQUE(user_id, project_id)
);

CREATE INDEX idx_favorites_user ON favorites(user_id);
```
```

Table: `saved\_searches`

```
```sql
CREATE TABLE saved_searches (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 name TEXT NOT NULL, -- user-given label e.g. "3BHK in Kondapur"
 filters JSONB NOT NULL, -- serialized filter state
 last_run_at TIMESTAMPTZ,
 result_count INTEGER,
 created_at TIMESTAMPTZ DEFAULT NOW(),
 updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Example filters JSONB:
-- {
-- "pinCodes": ["500075"],
-- "district": "Rangareddy",
-- "minFlats": 100,
-- "maxFlats": 500,
-- "projectStatus": "Ongoing",
-- "sortBy": "bookingPercent"
-- }
```



```
CREATE INDEX idx_saved_searches_user ON saved_searches(user_id);
...
```

Table: `comparison\_results`

```
```sql
CREATE TABLE comparison_results (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name TEXT, -- optional label
  project_ids TEXT[] NOT NULL, -- array of project ids being compared
  snapshot JSONB, -- optional: snapshot of comparison data at save time
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_comparisons_user ON comparison_results(user_id);
...
```

API Endpoints — Auth (`/api/auth`)

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	Public	Sign up with email + password
POST	`/api/auth/login`	Public	Sign in, returns access + refresh tokens
POST	`/api/auth/logout`	User JWT	Revoke refresh token
POST	`/api/auth/refresh`	Refresh token	Get new access token
POST	`/api/auth/forgot-password`	Public	Send reset email
POST	`/api/auth/reset-password`	Reset token	Set new password
POST	`/api/auth/verify-email`	Verify token	Confirm email address
POST	`/api/auth/resend-verification`	Public	Resend verification email

API Endpoints — User Profile (`/api/user`)

Method	Endpoint	Auth	Description
GET	/api/user/profile	User JWT	Get own profile
PUT	/api/user/profile	User JWT	Update name, mobile, avatar
PUT	/api/user/change-password	User JWT	Change password (requires current)
DELETE	/api/user/account	User JWT	Soft-delete own account

API Endpoints — Saved Properties (/api/user/saved-properties)

Method	Endpoint	Auth	Description
GET	/api/user/saved-properties	User JWT	List all saved properties
POST	/api/user/saved-properties	User JWT	Save a property { projectId, notes }
DELETE	/api/user/saved-properties/{projectId}	User JWT	Remove a saved property
GET	/api/user/saved-properties/{projectId}/exists	User JWT	Check if already saved

API Endpoints — Favorites (/api/user/favorites)

Method	Endpoint	Auth	Description
GET	/api/user/favorites	User JWT	List all favorites
POST	/api/user/favorites	User JWT	Add favorite { projectId }
DELETE	/api/user/favorites/{projectId}	User JWT	Remove favorite
GET	/api/user/favorites/{projectId}/exists	User JWT	Check if favorited

API Endpoints — Saved Searches (/api/user/saved-searches)

Method	Endpoint	Auth	Description
GET	/api/user/saved-searches	User JWT	List all saved searches
POST	/api/user/saved-searches	User JWT	Save a search { name, filters }

PUT	`/api/user/saved-searches/{id}`	User JWT	Rename or update filters
DELETE	`/api/user/saved-searches/{id}`	User JWT	Delete a saved search
POST	`/api/user/saved-searches/{id}/run`	User JWT	Re-run the search, returns results

API Endpoints — Comparison Results (`/api/user/comparisons`)

Method	Endpoint	Auth	Description
GET	`/api/user/comparisons`	User JWT	List saved comparisons
POST	`/api/user/comparisons`	User JWT	Save comparison `{ name?, projectIds[] }`
GET	`/api/user/comparisons/{id}`	User JWT	Get a specific comparison
DELETE	`/api/user/comparisons/{id}`	User JWT	Delete a comparison

JWT Token Strategy

...

Access Token

- Algorithm: HS256 (existing) → upgrade to RS256 in production
- Expiry: 15 minutes
- Claims: sub (user UUID), email, role, name

Refresh Token

- Opaque random token (UUID v4 → stored as SHA-256 hash in DB)
- Expiry: 30 days
- Sent as HttpOnly cookie OR Bearer (configurable)
- Rotation: new refresh token issued on each /refresh call
- Old token revoked immediately (one-time use)

...

Low-Level Design — Key .NET Classes

DTOs

```
```csharp
// Registration
public record RegisterRequestDto(
 string FullName,
 string Email,
 string Password,
 string? Mobile
);

// Login response
public record AuthResponseDto(
 string AccessToken,
 string RefreshToken,
 DateTime ExpiresAt,
 UserProfileDto User
);

// User profile
public record UserProfileDto(
 Guid Id,
 string FullName,
 string Email,
 string? Mobile,
 string? AvatarUrl,
 bool IsVerified,
 DateTime CreatedAt
);

// Saved property
public record SavedPropertyDto(
 Guid Id,
 string ProjectId,
 string ProjectName,
 string? Locality,
```

```

string? District,
string? ProjectStatus,
string? Notes,
DateTime SavedAt
);

// Saved search
public record SavedSearchDto(
 Guid Id,
 string Name,
 JsonElement Filters,
 int? ResultCount,
 DateTime? LastRunAt,
 DateTime CreatedAt
);

// Comparison
public record ComparisonResultDto(
 Guid Id,
 string? Name,
 string[] ProjectIds,
 DateTime CreatedAt
);
```

```

UserRepository interface

```

```csharp
public interface IUserRepository
{
 Task<User?> GetByEmailAsync(string email);
 Task<User?> GetByIdAsync(Guid id);
 Task<User> CreateAsync(User user);
 Task UpdateAsync(User user);
 Task<bool> EmailExistsAsync(string email);

 // Tokens

```

```

Task<RefreshToken?> GetRefreshTokenAsync(string tokenHash);
Task SaveRefreshTokenAsync(RefreshToken token);
Task RevokeRefreshTokenAsync(string tokenHash);
Task<PasswordResetToken?> GetPasswordResetTokenAsync(string tokenHash);
Task SavePasswordResetTokenAsync>PasswordResetToken token);
Task MarkPasswordResetTokenUsedAsync(string tokenHash);
}

```

```

public interface IUserDataRepository
{
 // Saved Properties
 Task<IEnumerable<SavedProperty>> GetSavedPropertiesAsync(Guid userId);
 Task<SavedProperty?> GetSavedPropertyAsync(Guid userId, string projectId);
 Task<SavedProperty> AddSavedPropertyAsync(SavedProperty item);
 Task RemoveSavedPropertyAsync(Guid userId, string projectId);

 // Favorites
 Task<IEnumerable<Favorite>> GetFavoritesAsync(Guid userId);
 Task<bool> IsFavoritedAsync(Guid userId, string projectId);
 Task AddFavoriteAsync(Guid userId, string projectId);
 Task RemoveFavoriteAsync(Guid userId, string projectId);

 // Saved Searches
 Task<IEnumerable<SavedSearch>> GetSavedSearchesAsync(Guid userId);
 Task<SavedSearch> AddSavedSearchAsync(SavedSearch search);
 Task<SavedSearch?> UpdateSavedSearchAsync(Guid userId, Guid searchId, string
name, JsonDocument filters);
 Task DeleteSavedSearchAsync(Guid userId, Guid searchId);

 // Comparisons
 Task<IEnumerable<ComparisonResult>> GetComparisonsAsync(Guid userId);
 Task<ComparisonResult?> GetComparisonAsync(Guid userId, Guid comparisonId);
 Task<ComparisonResult> AddComparisonAsync(ComparisonResult comparison);
 Task DeleteComparisonAsync(Guid userId, Guid comparisonId);
}
...

```

Password Reset Flow

...

1. POST /api/auth/forgot-password { email }
  - Generate cryptographically random token (32 bytes)
  - Store SHA-256(token) in password\_reset\_tokens with 1hr expiry
  - Send email with link: [https://app/reset-password?token=<raw\\_token>](https://app/reset-password?token=<raw_token>)
  - Always return 200 (prevent email enumeration)

2. POST /api/auth/reset-password { token, newPassword }
  - Hash incoming token → look up in DB
  - Verify not expired, not used
  - BCrypt hash new password → update users.password\_hash
  - Mark token used\_at = NOW()
  - Revoke all refresh tokens for that user (force re-login everywhere)

...

## Email Verification Flow

...

1. On register → generate verify token → send email
2. POST /api/auth/verify-email { token }
  - Validate token → set users.is\_verified = true, email\_verified\_at = NOW()
3. Endpoints that require verified email return 403 with:  
{ error: "email\_not\_verified", message: "Please verify your email first" }

...

---

## Security Considerations

| Concern | Mitigation |

|---|---|

| Password storage | BCrypt with cost factor 12 |

| Token storage | Only hashes stored in DB, raw tokens only in transit |

| Email enumeration on forgot-password | Always return 200 regardless of email existence |

Brute force on login	Existing IP rate limiting (5 req/min on `/api/login`) — extend to `/api/auth/login`
JWT secret	Rotate to RS256 asymmetric key in production
SQL injection	Parameterized queries via Dapper only
XSS in user inputs	Existing `InputSanitizer` applied to all user-provided strings
Account takeover	Refresh token rotation + revoke-all on password reset

---

## Updated Migration Phases

Phase	Adds
1	PostgreSQL + scrapers write to DB
2	.NET reads projects from DB
\*\*2b\*\*	\*\*User tables: `users`, `refresh\_tokens`, `password\_reset\_tokens`, `email\_verification\_tokens`\*\*
\*\*2c\*\*	\*\*Auth endpoints: register, login, logout, refresh, forgot/reset password, verify email\*\*
\*\*2d\*\*	\*\*User feature tables: `saved\_properties`, `favorites`, `saved\_searches`, `comparison\_results`\*\*
\*\*2e\*\*	\*\*User feature endpoints + Angular integration\*\*
3	Remove JSON file writing
4	Redis cache (optional)

---

## NuGet Packages to Add (Auth-specific)

```
```xml
<PackageReference Include="BCrypt.Net-Next" Version="4.0.3" /> <!-- already used -->

<PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="7.5.0" />
<!-- already used -->

<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer"
Version="8.0.0" />

<PackageReference Include="MailKit" Version="4.7.1" /> <!-- for SMTP email -->
```
```



---

## Correctness Properties

\*A property is a characteristic or behavior that should hold true across all valid executions of a system — essentially, a formal statement about what the system should do. Properties serve as the bridge between human-readable specifications and machine-verifiable correctness guarantees.\*

### Property 1: Scraper Upsert Idempotency

\*For any\* project data payload, running the RERA scraper twice for the same project ID must result in exactly one row in the `projects` table, with the `raw\_data` field reflecting the most recent write and `updated\_at` updated.

**\*\*Validates: Requirements 2.1\*\***

---

### Property 2: SRO Insert Idempotency

\*For any\* SRO transaction batch, inserting the same batch into `sro\_transactions` a second time must produce the same row count as inserting it once (no duplicates, `ON CONFLICT DO NOTHING`).

**\*\*Validates: Requirements 2.2\*\***

---

### Property 3: Unit Rates Replace Idempotency

\*For any\* set of unit rate data, running the RR scraper N times must produce the same final state in `unit\_rates` as running it once (truncate + re-insert semantics).

**\*\*Validates: Requirements 2.3\*\***

---

## Property 4: Scrape Run Status Consistency

\*For any\* scraper execution outcome, the terminal ``status`` value persisted in ``scrape_runs`` must match the actual outcome: ``'completed'`` when the run succeeded without fatal error, or ``'failed'`` when a fatal error occurred.

**\*\*Validates: Requirements 2.7, 2.8, 2.9\*\***

---

## Property 5: Repository Implementation Parity

\*For any\* project identifier present in both the file-based data source and PostgreSQL after Phase 1, the response from ``PostgresProjectRepository.GetProjectByIdAsync(id)`` must produce field values equivalent to those from the file-based ``ProjectRepository`` for the same ``id``.

**\*\*Validates: Requirements 3.3, 4.1, 6.3\*\***

---

## Property 6: Filter Correctness

\*For any\* ``pinCode`` value, every project returned by ``GetProjectsByPinCodeAsync(pinCode)`` must have a ``pin_code`` column value equal to the supplied ``pinCode``, and no project with a different ``pin_code`` must appear in the results.

**\*\*Validates: Requirements 4.5\*\***

---

## Property 7: Raw Data Round-Trip Fidelity

\*For any\* scraped project, the JSONB ``raw_data`` written to PostgreSQL by the Python scraper, when read back and deserialized by the .NET API, must produce a JSON structure equivalent to the original ``view_page_data.json`` content for that project.

**\*\*Validates: Requirements 6.2\*\***

---

## Property 8: Registration Produces Unverified Active User

\*For any\* valid registration payload (varying names, emails, and optional mobile numbers), a successful `POST /api/auth/register` must create a user record with `is_verified = false` and `is_active = true`.

**\*\*Validates: Requirements 7.1\*\***

---

## Property 9: Password Never Stored in Plaintext

\*For any\* password string submitted during registration or password change, the value stored in `users.password_hash` must not equal the plaintext password, and the `BCrypt Verify(plaintext, stored_hash)` function must return `true`.

**\*\*Validates: Requirements 7.2, 18.1\*\***

---

## Property 10: Email Uniqueness Enforced

\*For any\* email address that already exists in the `users` table, a subsequent `POST /api/auth/register` with that same email must return `409 Conflict` without creating a duplicate user record.

**\*\*Validates: Requirements 7.3\*\***

---

## Property 11: Token Hash-Only Storage

\*For any\* token generated by the system (refresh token, password reset token, or email verification token), the value stored in the database must be the SHA-256 hash of the raw token and must not equal the raw token value itself.

**\*\*Validates: Requirements 8.6, 9.3, 11.6, 18.2\*\***

---

## Property 12: Login Returns Structurally Valid Auth Response

\*For any\* registered user with verified credentials, a `POST /api/auth/login` must return an `AuthResponseDto` containing a non-empty `accessToken` (decodable as a JWT with claims `sub`, `email`, `role`, `name`), a non-empty `refreshToken`, a future `expiresAt` timestamp, and a `UserProfileDto` with a matching `email`.

**\*\*Validates: Requirements 9.1, 9.2\*\***

---

## Property 13: Refresh Token Rotation

\*For any\* valid, unexpired, unrevoked refresh token, a `POST /api/auth/refresh` must issue a new access token and new refresh token, and must mark the original refresh token as revoked so that a subsequent refresh attempt with the original token returns `401 Unauthorized`.

**\*\*Validates: Requirements 10.1, 10.3\*\***

---

## Property 14: Logout Permanently Invalidates Refresh Token

\*For any\* authenticated user session, after a successful `POST /api/auth/logout`, all subsequent `POST /api/auth/refresh` requests using the revoked refresh token must return `401 Unauthorized`.

**\*\*Validates: Requirements 10.4, 10.5\*\***

---

## Property 15: Forgot Password Non-Enumeration

\*For any\* email address, whether registered or not, `POST /api/auth/forgot-password` must return `200 OK`.

**\*\*Validates: Requirements 11.1, 18.4\*\***

---

## Property 16: Password Reset Round-Trip

*\*For any\** user with a valid, unexpired password reset token and a new password, after a successful `POST /api/auth/reset-password``, a subsequent `POST /api/auth/login`` with the new password must succeed, and a login attempt with the old password must return `401 Unauthorized``.

**\*\*Validates: Requirements 11.3\*\***

---

## Property 17: Password Reset Revokes All Sessions

*\*For any\** user who successfully resets their password, all refresh tokens that existed before the reset must be revoked, such that refresh requests using any of those tokens return `401 Unauthorized``.

**\*\*Validates: Requirements 11.4, 18.7\*\***

---

## Property 18: Profile Update Round-Trip

*\*For any\** authenticated user and any valid profile update payload (``fullName``, ``mobile``, ``avatarUrl``), after a successful `PUT /api/user/profile``, a subsequent `GET /api/user/profile`` must return the updated values for the changed fields.

**\*\*Validates: Requirements 12.1, 12.2\*\***

---

## Property 19: Deactivated User Access Revocation

\*For any\* user whose `is\_active` is set to `false`, all authenticated requests to `/api/user/\*` endpoints using tokens issued before or after deactivation must return `401 Unauthorized`.

**\*\*Validates: Requirements 12.5, 12.7\*\***

---

## Property 20: Saved Properties Add-Exists-Delete Round-Trip

\*For any\* authenticated user and any valid `projectId`, after saving a property and then deleting it, the `GET /api/user/saved-properties/{projectId}/exists` endpoint must return `false`, and the `GET /api/user/saved-properties` list must not contain the deleted entry.

**\*\*Validates: Requirements 13.1, 13.3, 13.4, 13.5\*\***

---

## Property 21: Saved Properties Uniqueness Invariant

\*For any\* authenticated user and any `projectId` already present in that user's saved properties, a second `POST /api/user/saved-properties` with the same `projectId` must return `409 Conflict` without creating a duplicate entry.

**\*\*Validates: Requirements 13.2\*\***

---

## Property 22: Favorites Add-Exists-Delete Round-Trip

\*For any\* authenticated user and any valid `projectId`, after adding a favorite and then removing it, the `GET /api/user/favorites/{projectId}/exists` endpoint must return `false`, and the favorites list must not contain the removed entry.

**\*\*Validates: Requirements 14.1, 14.3, 14.4, 14.5\*\***

---

## Property 23: Favorites Uniqueness Invariant

\*For any\* authenticated user and any `projectId` already in that user's favorites, a second `POST /api/user/favorites` with the same `projectId` must return `409 Conflict` without creating a duplicate entry.

**\*\*Validates: Requirements 14.2\*\***

---

## Property 24: Saved Search Filters Round-Trip

\*For any\* saved search with a filters JSONB object, the `filters` field returned by `GET /api/user/saved-searches` must be structurally equivalent to the filters object that was submitted during `POST /api/user/saved-searches`, preserving all fields including `pinCodes`, `district`, `minFlats`, `maxFlats`, `projectStatus`, and `sortBy`.

**\*\*Validates: Requirements 15.1, 15.8\*\***

---

## Property 25: Saved Search Run Filter Correctness

\*For any\* saved search with filter criteria, `POST /api/user/saved-searches/{id}/run` must return only projects that satisfy all of the saved filter criteria, and must not return any project that fails any one of those criteria.

**\*\*Validates: Requirements 15.5\*\***

---

## Property 26: Saved Search Update Round-Trip

\*For any\* saved search, after a `PUT /api/user/saved-searches/{id}` with new `name` and/or `filters`, a subsequent `GET /api/user/saved-searches` must return the updated `name` and `filters` for that entry and `updated\_at` must be later than `created\_at`.

**\*\*Validates: Requirements 15.3\*\***

---

## Property 27: Comparison Create-Retrieve Round-Trip

*\*For any\** array of 2 or more valid project IDs, after a `POST /api/user/comparisons`, a `GET /api/user/comparisons/{id}` must return a `ComparisonResultDto` with a `projectIds` array containing exactly the same project IDs that were submitted.

**\*\*Validates: Requirements 16.1, 16.3\*\***

---

## Property 28: User Data Isolation

*\*For any\** two distinct authenticated users A and B, user A must not be able to read, modify, or delete saved properties, favorites, saved searches, or comparison results that belong to user B — all such cross-user requests must return `403 Forbidden`.

**\*\*Validates: Requirements 17.5\*\***

---

## Property 29: Unverified User Access Restriction

*\*For any\** user whose `is_verified` is `false`, requests to saved-properties, favorites, saved-searches, and comparisons endpoints must return `403 Forbidden` with error code `email_not_verified`.

**\*\*Validates: Requirements 17.4\*\***